torchvision RetinaNet (ResNet-50 + FPN): Official Recipe vs Our Implementation

Official source: pytorch/vision, references/detection (train.py and engine.py on the main branch, README RetinaNet command, model docs page). Project: TBX11K lesion localization (Active TB, Obsolete TB). Re-verified 10 November 2026.

How to read this document

Verification status. **READ in official file** means the value was read in the official file during this work. **READ in a copy / older version** means the behaviour was read in an older torchvision source page or in a copy of the file outside the official repo, so it must still be confirmed against the installed version. **FROM MEMORY - confirm** means the file could not be read, so the value is my recollection and must be checked against the real file before anyone claims it. **Change type.** IDENTICAL = same as official. DATASET = needed because the data is different. SYSTEM = needed because of the hardware (one RTX 4050 with 6 GB). PROJECT = a project-wide protocol choice (70 epochs, 512 x 512 input). ADDITION = added on top of the official script (best-checkpoint saving).
Planned runs. Run A (first): from scratch, to match the other four models in this project. Run B: ImageNet-pretrained backbone, as in the official recipe. Everything else in the table is the same for both runs.

Setting	Official torchvision recipe	Verification status	What we will implement	Change type
A. Model and data				
Model	torchvision.models.detection.retinanet_resnet50_fpn (v1). Training command uses --model retinanet_resnet50_fpn.	**READ in official file** README command; docs page	Same model (v1, not v2).	**IDENTICAL**
Dataset / classes	COCO. num_classes comes from the dataset (91, including background).	**READ in official file** train.py; docs say num_classes includes background	TBX11K TB images. num_classes = 3 (background + Active TB + Obsolete TB).	**DATASET**
Training images	Images without annotations are removed from the COCO training set (train split only; the validation split is not filtered).	**READ in a copy / older version** get_coco in coco_utils.py, read in a copy hosted in a Hugging Face Space. The official file itself was not reachable.	Only images with at least one box (the TB images) are used for training and validation.	**DATASET**
Backbone initialization	ImageNet ResNet-50. Note: train.py has --weights-backbone default None and passes weights_backbone=args.weights_backbone to get_model unconditionally, so None is passed explicitly. The README RetinaNet command passes no weights flag. The model builder's own default is ResNet50_Weights.IMAGENET1K_V1 (docs page).	**READ in official file** train.py (model-creation call and argparse); docs page. In the older 0.10-era source the backbone default was pretrained_backbone=True.	**Two runs, same settings otherwise.** Run A (first): from scratch, weights=None and weights_backbone=None (random backbone). Run B: weights_backbone=ResNet50_Weights.IMAGENET1K_V1 passed explicitly (the official recipe's starting point). The FPN and heads start random in both.	**PROJECT**
Trainable backbone layers	--trainable-backbone-layers default None. With an ImageNet backbone the builder uses 3 (stem and layer1 frozen).	**READ in official file** train.py; docs page. The rule used when the model is NOT pretrained is in _validate_trainable_layers, whose body was not visible: believed to force all 5 layers trainable - confirm.	Run B: 3. Run A: all layers trainable (a frozen random layer cannot learn). Confirm in the installed torchvision.	**PROJECT**
Backbone normalization layers	FrozenBatchNorm2d when the backbone is pretrained. For a non-pretrained model, newer torchvision builders use ordinary trainable BatchNorm2d; the older 0.10-era source passed no norm_layer argument, so it kept its own default.	**FROM MEMORY - confirm** builder logic not visible on the older source page. Must be read in the installed torchvision.	Run B: frozen BatchNorm (official). Run A: trainable BatchNorm2d as the installed builder gives. Note: at batch 4 BatchNorm statistics are noisy; report this as a limitation. Do not swap in GroupNorm unless you decide to and label it a deviation.	**PROJECT**

Setting	Official torchvision recipe	Verification status	What we will implement	Change type
Input size	Model transform defaults: min_size 800, max_size 1333.	**READ in a copy / older version** retinanet.py as rendered in the older torchvision docs source (0.10-era; the version is not stated on the page). Confirm against the installed version.	512 x 512 (min_size=512, max_size=512), as for every model in this project.	**PROJECT**
Normalization	image_mean and image_std default to the ImageNet values [0.485, 0.456, 0.406] and [0.229, 0.224, 0.225] inside the model transform.	**READ in a copy / older version** older torchvision source	Same (left to the model transform).	**IDENTICAL**
Anchors, FPN, loss, matching	Anchor sizes 32, 64, 128, 256, 512 per level, 3 scales each, aspect ratios 0.5 / 1 / 2 (9 anchors per location). FPN levels P3-P7 (returned_layers 2, 3, 4 plus LastLevelP6P7). Classification: sigmoid focal loss. Box loss: L1 (sum, divided by the number of foreground anchors). Matching: IoU 0.5 foreground, 0.4 background.	**READ in a copy / older version** older torchvision source. The focal-loss alpha and gamma defaults were not visible there (believed 0.25 and 2): confirm.	Same defaults, not modified.	**IDENTICAL**
Loss function	Built into the model, not set by train.py (train.py has no loss argument). Classification: sigmoid focal loss, summed over anchors and divided by max(1, number of foreground anchors). Box regression: L1 loss, summed over foreground anchors and divided by max(1, number of foreground anchors). Both are averaged over the images in the batch and returned as a dictionary of two terms; the reference engine.py adds the terms with equal weight.	**READ in a copy / older version** RetinaNetClassificationHead and RetinaNetRegressionHead compute_loss in the older torchvision source (0.10-era). Focal-loss alpha 0.25 and gamma 2 are the torchvision.ops defaults but were not visible there: confirm.	The same built-in loss, not modified (no custom loss, no class weights).	**IDENTICAL**
Post-processing	Defaults: score threshold 0.05, NMS IoU 0.5, 300 detections per image, 1000 top candidates per level.	**READ in a copy / older version** older torchvision source	Same defaults.	**IDENTICAL**
B. Optimization and training loop				
Optimizer	sgd (--opt default).	**READ in official file** train.py	SGD.	**IDENTICAL**
Base learning rate	0.01 (the RetinaNet command passes --lr 0.01). train.py default is 0.02, and its help text says: "initial learning rate, 0.02 is the default value for training on 8 gpus and 2 images_per_gpu". The file's docstring adds: "the learning rate should be changed to 0.02/8*$NGPU".	**READ in official file** README command; train.py help text and docstring	0.0025 = 0.01 x 4 / 16: linear scaling with the batch size (Goyal et al., 2017, Section 2.1), which is also what the torchvision docstring advises. See the learning-rate section.	**SYSTEM**
Momentum	0.9	**READ in official file** train.py	0.9	**IDENTICAL**
Weight decay	1e-4. --norm-weight-decay default None, so all trainable parameters share one parameter group.	**READ in official file** train.py	1e-4, one parameter group.	**IDENTICAL**
LR schedule	MultiStepLR, milestones [16, 22] (--lr-steps 16 22), gamma 0.1.	**READ in official file** train.py; README command	MultiStepLR, milestones [43, 59], gamma 0.1. These are the official 16 and 22 of 26 epochs scaled to 70 epochs (x 70 / 26, rounded).	**PROJECT**

Setting	Official torchvision recipe	Verification status	What we will implement	Change type
Warmup	Epoch 0 only. A separate LinearLR with start_factor 1/1000 and total_iters = min(1000, len(data_loader) - 1), stepped after every batch. The main MultiStepLR is stepped once per epoch, after the epoch.	**READ in official file** engine.py and train.py	Same code. With about 560 training images at batch 4 this is about 139 iterations (depends on the real image count).	**IDENTICAL**
Epochs	26	**READ in official file** train.py default; README command	70, no early stopping.	**PROJECT**
Batch size	2 images per GPU (default), 8 GPUs, so a total batch of 16 (8x V100 per README).	**READ in official file** train.py; README	4 on one GPU.	**SYSTEM**
Gradient clipping	None. No clipping call appears in engine.py.	**READ in official file** engine.py	None.	**IDENTICAL**
Mixed precision	Off by default (--amp flag).	**READ in official file** train.py; engine.py	AMP on (RTX 4050, 6 GB).	**SYSTEM**
Data loading workers	4	**READ in official file** train.py	2	**SYSTEM**
Augmentation	--data-augmentation default hflip (horizontal flip).	**READ in official file** train.py default. The flip probability 0.5 is in presets.py, which was not read.	Random horizontal flip, probability 0.5, boxes flipped too. Nothing else.	**IDENTICAL**
Samplers	RandomSampler for training. --aspect-ratio-group-factor 3 uses GroupedBatchSampler. Test loader: batch size 1, SequentialSampler.	**READ in official file** train.py	Same. (Chest X-rays are nearly square, so the aspect-ratio grouping changes little.)	**IDENTICAL**
Distributed training	DistributedDataParallel on 8 GPUs. --sync-bn off by default.	**READ in official file** train.py	Single GPU, no distributed launch.	**SYSTEM**
Random seed	No seed argument in train.py.	**READ in official file** train.py argparse list	Seed 42.	**SYSTEM**
Early stopping	None.	**READ in official file** train.py	None.	**IDENTICAL**
Checkpointing	Saves model_{epoch}.pth every epoch plus checkpoint.pth. No best-model selection.	**READ in official file** train.py	Same, plus best_model.pth chosen by validation COCO AP (0.5:0.95, mean of the two classes) and last_checkpoint.pth.	**ADDITION**
Evaluation	evaluate() after every epoch with CocoEvaluator (bbox).	**READ in official file** train.py; engine.py	Same evaluator, run on the validation split each epoch. Test split evaluated once with the best checkpoint, plus the project's box-level threshold protocol.	**PROJECT**

Learning rate: did the official recipe use scaling?

Question	Answer
Does train.py scale the learning rate automatically?	No. In the code, the optimizer is built with lr=args.lr as given. No line multiplies it by the number of GPUs, the world size or the batch size.
Does the official recipe tell you to scale it?	**Yes, manually.** The docstring of train.py says the learning rate "should be changed to 0.02/8*$NGPU" when the number of GPUs differs from 8 (at 2 images per GPU). The --lr help text says the same thing in another form: "0.02 is the default value for training on 8 gpus and 2 images_per_gpu". So the official guidance is linear scaling of the LR with the total batch size. It is advice in the file, not something the code does for you.
What LR does the RetinaNet recipe use?	The RetinaNet README command passes --lr 0.01 on 8 GPUs x 2 images (batch 16). That is half of the 0.02 default. The docstring formula is written for the 0.02 default, so for RetinaNet the same rule is applied to its own base: 0.01/8 per GPU (2 images each).
Where does the scaling rule come from?	Goyal, Dollar, Girshick, Noordhuis, Wesolowski, Kyrola, Tulloch, Jia and He, "Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour", arXiv:1706.02677 (2017). Section 2.1, "Learning Rates for Large Minibatches": *"Linear Scaling Rule: When the minibatch size is multiplied by k, multiply the learning rate by k."* The paper keeps all other hyper-parameters (weight decay, etc.) unchanged, and pairs the rule with a gradual warmup (Section 2.2). The text could not give the page number: open the PDF and note it before citing a page. The torchvision docstring does not cite this paper (not visible in the lines read), so linking the two is our inference. Safe wording: "the learning rate was scaled linearly with the batch size (Goyal et al., 2017), as advised in torchvision's detection reference script". Do not claim the exact technique of the paper: the paper concerns ImageNet classification with large batches and its own warmup, whereas the warmup here is torchvision's (1/1000, epoch 0 only).
What should we do at batch 4 on one GPU?	**Scale linearly, as the official docstring advises.** LR = 0.01 x 4 / 16 = **0.0025**. Same result by the docstring rule: 4 images per step is the equivalent of 2 GPUs x 2 images, so LR = 0.01/8 x 2 = 0.0025. It is also the rule used for the EfficientNet run in this project. **Alternative that needs no scaling:** accumulate gradients over 4 steps (effective batch 16) and keep LR 0.01. With frozen BatchNorm this is nearly equivalent; the loss normalization differs slightly. Do not keep 0.01 at a true batch of 4 without saying so: each image would then have 4 times the official influence per step, and RetinaNet's focal loss can become unstable at high LR.

Numbers derived for our run

Quantity	Formula	Value
Base LR at batch 4	0.01 x 4 / 16	0.0025
LR drop 1	16 x 70 / 26 = 43.08, rounded	epoch 43
LR drop 2	22 x 70 / 26 = 59.23, rounded	epoch 59
Warmup iterations	min(1000, iterations per epoch - 1); about 560 / 4 = 140 iterations per epoch	about 139 (confirm with the real image count)

Not yet verified (must be confirmed before claiming "identical to official")

- presets.py: the exact transforms for the hflip policy (flip probability 0.5, dtype conversion) could not be read; the 0.5 probability is from memory.
- group_by_aspect_ratio.py (GroupedBatchSampler internals) was not read.
- The body of _validate_trainable_layers and the BatchNorm choice in the model builder (what happens to trainable layers and normalization when no pretrained weights are used) were not visible: they are version-dependent and must be read in the installed torchvision before the from-scratch run.
- The focal-loss alpha and gamma defaults (torchvision.ops.sigmoid_focal_loss) could not be opened.
- The page number of the linear-scaling rule in Goyal et al. could not be read from the text (Section 2.1 is confirmed).
- The model defaults (input size, thresholds, anchors, losses) were read only in an older torchvision source page (0.10-era) and the coco_utils behaviour only in a copy hosted outside the official repo. The focal-loss alpha and gamma defaults were not visible. All of these must be confirmed against the files that match the torchvision version installed on the training machine.
- The current references/detection README on GitHub could not be read; only an older copy (the RetinaNet command) was.

Sources

- pytorch/vision, references/detection/train.py (main branch): github.com/pytorch/vision/blob/main/references/detection/train.py
- pytorch/vision, references/detection/engine.py (main branch): github.com/pytorch/vision/blob/main/references/detection/engine.py
- RetinaNet training command from the references/detection README (older copy and the commit that added RetinaNet): gitee.com/wqy11888/pytorch-vision-faster-rcnn and the pytorch/vision commit 5bb81c8
- Older torchvision source of retinanet.py (0.10-era): pytorch.org/vision/0.10/_modules/torchvision/models/detection/retinanet.html
- coco_utils.py as hosted in a copy (not the official repo): huggingface.co/spaces/vorkna/MaskRCNN-FishEye/blob/main/coco_utils.py
- torchvision documentation, retinanet_resnet50_fpn: docs.pytorch.org/vision/main/models/generated/torchvision.models.detection.retinanet_resnet50_fpn.html
- Goyal et al., Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour, arXiv:1706.02677 (read at arxiv.org/pdf/1706.02677v2 and ar5iv.arxiv.org/html/1706.02677)